

学习日志 · 云计算转行

这份文件怎么用

- 1. 我按《云计算6个月转行计划》逐课带你走，每课只推一小步，不提前灌后面内容。
- 2. 你在 Ubuntu 里跑我给的命令，把输出原样贴回来（贴错、报错都行，报错更有价值）。
- 3. 我把解读写进本文件的「我的解读」里，把卡点写进「卡点」，再决定下一课。
- 4. 你随时可以在本文件里补自己的话。

教材： 云计算6个月转行计划.md

怎么学（每课的标准动作）

一句话结论：先跟做一遍，再关掉文档自己做一遍 —— 两者都要，顺序不能反，但绝不允许"边看边敲"。

三段式（每课都按这个走）

段	做什么	时长	过关标准
① 跟做	照讲义敲一遍，每条命令敲之前先说出"我预期看到什么"	25~35 min	命令全跑通（报错也算，但必须看到报错原文）
② 单点闭卷	合上讲义，一次只闭卷 3 条命令（不是整课）；忘了就看一眼 → 立刻关掉 → 重新敲	10~15 min	给一个场景能说出该用哪条命令（参数允许查）
③ 变式 + 故意改坏	换参数、故意写错一个值、看报错、再修好	15~20 min	能说出这个报错到底在说什么

修正（2026-09-22）：上一版把第 ② 段写成"合上讲义把整课重做一遍、要完成 80%"。那是我写错了，不是你的问题。一条新命令敲过一遍就记不住，是正常现象。正确做法：闭卷的单位是一条命令，不是一整课；第一遍的目标是"知道有这条命令、知道它解决什么问题"，不是背参数。

② 才是真正长本事的地方。① 只让你"知道有这回事"，② 才把它长在自己身上。

三条铁律

- 1. 不许边看边敲。边看边敲走的是"眼睛 → 手指"的抄写回路；而面试和故障现场需要的是"问题 → 命令"的检索回路。跟着视频敲完就忘，原因就在这里。
- 2. 先预测，再执行。敲之前先写一句"我预期....."再回车。对了 = 验证了理解；错了 = 抓到一个认知盲点（比答对更值钱）。
- 3. 卡住超过 5 分钟就回去看，但看完必须关掉重敲一遍。不许"看着答案把手放上去"。

哪些能复制粘贴，哪些必须手打

类型	例子	处理方式
要形成肌肉记忆的	cd/ls/grep/systemctl/journalctl/docker ps/ss	必须手打，练到不用看键盘
一次性长命令	apt install 一长串、echo "deb ..."、多行 heredoc	可以粘贴，但要逐段读一遍再回车
危险命令	rm -rf、mkfs、swapoff、挂载	一律手打，且先 lsblk/df -h 确认目标

每课结束必须留痕（否则等于白学）

- 把输出贴回 学习日志.md 对应课的「我的实测输出」（贴报错原文比贴成功更有价值）
- 笔记三件套：今天学了什么 / 报错原文 / 怎么解的
- 用面试题库做口述自测：先只看题、张嘴说，说完再对答案

分支判断（不用纠结）

情况	怎么做
这个命令第一次见	跟做（①），照着敲
这个命令已经跟做过一次	直接关文档自己做（②），不许再翻讲义
卡住 > 5 分钟	回去看答案 → 关掉 → 重新敲一遍
讲义标注 A 型日	输入日：看教程时间 ≤ 总时间 1/3，其余必须自己敲
讲义标注 B 型日	练习日：不许看教程，卡住只能查 man 和官方文档
一条命令闭卷 3 次仍想不起来	别硬背 —— 回来问我，通常是原理没懂

记忆三档（不是所有东西都要脱手）

新手最大的内耗，是要求自己把所有东西都背下来。实际运维不这样 —— 老手也天天查 man 和官方文档，他们记住的是遇到什么问题、该用哪个工具。

档	要求	例子（第 1/2/3 课）
必须脱手	不看资料能敲出骨架（每课约 5~8 条）	df -h、free -h、lsblk -f、ip -a、systemctl status、docker ps、docker run -d -p、id -nG
可查	知道有这个东​​西，用时查参数	dmidecode、lsof +l1、lsblk -d -o NAME,ROTA、journalctl -xeu、apt-mark hold
认识即可	看到能说出它干什么	/proc/cpuinfo、live-restore、docker info 的各种字段

闭卷阶梯：第一遍只需要到 L1

阶梯	闭卷时你必须能...	允许查什么	什么时候达到
L0 认识	看到命令能说出它是干什么的	随便查	第一遍
L1 会说	给一个场景，说出该用哪个命令	参数、选项全部可查	第一遍只需到这
L2 会敲	不看资料敲出命令骨架	--help / man	第二、三遍
L3 会用	组合多条命令排掉一个真实故障	官方文档	月末 / 检查点

别在第一遍就要求自己到 L3 —— 那是第 1 月结束时的目标，不是今晚的目标。

如果你只有 20 分钟（最低可完成版）

- 1. 读讲义一遍，只看要做什么（5 min）
- 2. 挑 3 条最重要的命令，手打一遍（5 min）
- 3. 关文档，重新敲这 3 条；忘了 → 看一眼 → 立刻关掉 → 重敲（7 min）
- 4. 在日志写一行：今天闭卷拿下这 3 条（3 min）

今天的完成 = 这 3 条能闭卷敲出来，不是整课全过。做完就是完成，不是没做完。
状态不好时就用这个版本 —— 做过 20 分钟，永远强过计划 2 小时却一步没动。

记不住怎么办：间隔重复，而不是当天死磕

- 当天不熟的，记进日志就好，不要当天死磕（死磕会制造我怎么这么笨的错觉）。
- 第二天开始前 10 分钟默写昨天的 3 条（计划里本来就有这一格）。
- 第 3 天、第 7 天再各过一遍 —— 记忆靠多次短接触长出来，不靠一次长时间硬背。
- 同一条命令闭卷 3 次还是想不起来，说明它不是记不住，而是原理没懂 —— 这时候回来问我，比继续硬背有效 10 倍。

每课的最小可用完成标准（做到就算过关）

课	今天算完成的最低线
第 1 课	能手打 df -h free -h lsblk -f，并能说出 df -h 和 df -i 看的東西不一样
第 2 课	能手打 docker ps docker run -d -p 8080:80 nginx curl -I localhost:8080，并能说出 id -nG 和 getent group docker 为什么不一致
第 3 课	能手打 systemctl status ssh ip -a，并能用密钥登录成功一次

每课的结构里必须有「本课必背清单」

每节课末尾都有一节 本课必背清单，分三档列出「★ 必须脱手 / 可以查 / 认识即可」，最后是必须能说出口的几句话。
复习一课只看那一节 —— 不用重读整课。学完新课时，先对着清单自测，再贴输出。

进度总览

周	课程	状态
W1 D1	第 1 课 · 先认清你的机器	进行中（待贴输出）
W1 D1-D2	第 2 课 · 装工具 + Docker	进行中（待贴输出）
W1 D2	第 3 课 · 让服务能被连上（SSH + 固定 IP）	进行中
W1 D2	第 4 课 · 建立"搞坏了能回来"的底气（备份）	待开始
W1 D3	第 5 课 · 文件系统：Linux 把东西放哪	待开始
W1 D4	第 6 课 · 纯动手：不碰鼠标完成 20 个文件操作	待开始
W1 D5	第 7 课 · 查找：find / grep	待开始

面试题库（每课一份）

用法：只看【问题清单】逐题口述并勾选 - []，再翻【参考答案】对照。答案只记结构：结论 → 为什么 → 命令/证据 → 加分点。
标记：★★★ 必背（面试高频 + 生产真遇到，有外部依据） / ★★ 理解 / 🛠 生产必会（企业天天用、面试少问） / ☆ 非面试题。
规则：每课上限 30 题；每组按星级降序（★★★ → ★★ → 🛠 → ☆，同级按题号升序）；题号永不重排。
规范与依据库：[面试题库/题库规范.md](#) —— 每写一课先看它。

课	面试题库	题量	★★★ 必背	🛠 生产必会	☆ 非面试题
第 1 课	先认清你的机器	24	10	1	1
第 2 课	装工具 + Docker	41	17	7	2
第 3 课	让服务能被连上（SSH + 固定 IP）	26	13	2	1

第 1 课 · 先认清你的机器

日期：2026-09-20 对应计划：W1 D1（含第 0 步的形态确认） 目标：搞清楚"我现在手上是什么机器"，并把"先看输出、再下结论"这个反射种下去。

这一课在解决什么

你以后会反复听到一句话：运维接手一台机器，第一件事永远不是装东西，是先搞清楚它是谁。

因为所有后续判断都依赖这几个事实：内核版本、内存多大、磁盘还剩多少、是不是跑在虚拟化里。这些不确定，后面出的每个错你都只能猜。

所以第 1 课我们一条业务命令都不敲，只做"体检"。

第 1 课只对应计划 1.7「环境验收标准」的第 1、2 条（系统确认、资源确认）。
剩下 7 条会在第 2~4 课补齐，其中 **第 8、9 条（形态专项、备份可用）必须在任何「故意搞坏」的练习之前做完。**

本课任务

任务 A：判断你是什么形态（2 条命令）

```
systemd-detect-virt
lsblk -f
```

预期输出长这样：

- `systemd-detect-virt` 输出 `none` → **物理机**（如果还装了 Windows，就是双系统）
- 输出 `kvm` / `vmware` / `oracle` (VirtualBox) / `microsoft` (Hyper-V) → **虚拟机**，这个单词就是你的宿主机虚拟化平台
- `lsblk -f` 会列出磁盘和分区，重点看：
- 除 Ubuntu 的分区外，有没有 `ntfs` 分区（有 = 双系统）
- Ubuntu 根分区挂载点是 `/`，容量多少
- `FSTYPE` 列是 `ext4`

补充讲解 • `lsblk -f` 到底在说什么

这是你对「硬盘 = 一块地」这个比喻的追问，结论记在这里，以后回看。

一、比喻的对应关系（逐项对上号）

比喻里的东西	实际是什么	在输出里的位置
一整块地	一块物理硬盘（或虚拟磁盘）	NAME 里 不带数字 的那行，如 <code>nvme0n1</code> 、 <code>sda</code>
地被切成几块	分区（partition）	NAME 里 带数字 的那行，如 <code>nvme0n1p3</code>
每块地上盖什么房	文件系统（filesystem）	FSTYPE 列： <code>ext4</code> / <code>ntfs</code> / <code>vfat</code> / <code>swap</code>
房子挂哪个门牌	挂载点（mountpoint）	MOUNTPOINTS 列： <code>/</code> 、 <code>/boot/efi</code>
房子能住多少	剩余容量、使用率	FSAVAIL、FSUSE%
房子的身份证号	UUID	UUID 列（写 <code>/etc/fstab</code> 时用它，别用设备名）

二、一份真实输出的样子（示例，你的会不一样）

双系统机器的典型输出（新版 `util-linux` 列名是 `MOUNTPOINTS`，老教程写 `MOUNTPOINT`，是同一回事）：

```
NAME      FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
nvme0n1
├─nvme0n1p1 vfat   FAT32   7A3B-1C4D                               511.7M   1% /boot/efi
├─nvme0n1p2 ntfs                                01D8C2A4F5B6E7C8
├─nvme0n1p3 ext4    1.0     5f2a9c31-8d4e-4b21-9a77-3c1e6f0b2d55  68.2G   28% /
└─nvme0n1p4 swap    1       a1b2c3d4-e5f6-7890-abcd-ef1234567890                                [SWAP]
```

树形缩进就是"分成了什么"的答案：└─┬─ 缩进在 `nvme0n1` 下面的，都是这块盘切出来的分区。数一数 `p1 p2 p3 p4` 就知道它被分成了 4 块。

- 设备名不带数字（`nvme0n1` / `sda` / `vda`）= 整块盘；带数字（`nvme0n1p3` / `sda3`）= 分区
- NVMe 固态叫 `nvme0n1p3`，SATA 盘或虚拟磁盘叫 `sda3`，只是接口不同，含义一样
- `MOUNTPOINTS` 为空 = 这块分区**没挂载**，系统现在用不上它（Windows 分区在 Ubuntu 里就是这种状态，或只读挂载）
- 看到 `loop0 loop1...不用管`：那是 `snap` 把文件当磁盘用，不是你真实的硬盘

三、必须分清：分区 ≠ 文件系统

- **分区 = 划地**：在分区表里登记"从第几 G 到第几 G 归我"
- **文件系统 = 盖房**：`mkfs.ext4` 才是在这块地上盖房
- 地可以空着不盖房 —— 这时 `FSTYPE` 就是空的，你看到的是"有地无房"
- **挂载 = 装门牌**：把房子接到 `/` 这棵目录树上你才走得进去。有房没门牌（没挂载），你在 `ls` 里是看不见它的

四、`lsblk` 的边界（它不告诉你的四件事）

你想知道	<code>lsblk</code> 能吗	该用什么
还有多少 未分配空间 （黑地）	不能	<code>sudo parted -l</code>
分区表是 MBR 还是 GPT	不能	<code>sudo fdisk -l</code> （看 <code>Disklabel type</code> ）
是固态还是机械	能（要加参数）	<code>lsblk -d -o NAME,ROTA,SIZE</code> ， <code>ROTA=0</code> 固态、1 机械
每个分区 谁吃掉了空间	不能	<code>df -h</code> （分区级）、 <code>du -sh ~/projects/*</code> （目录级）

记住这个分工：`lsblk` 看"盘的物理布局"，`df` 看"挂载后的用量"。排磁盘故障时两个都要跑。

五、跑完 `lsblk -f` 你要抓的三行

1. 根分区那一行（`MOUNTPOINTS` 是 `/` 的那行）：**容量和剩余**，抄进笔记

2. 有没有 ntfs 行：**有 = 双系统**，W2 破坏性练习前必须先做 Timeshift/快照
3. 整块盘的 NAME 和分区数量：第 1.5④ 节扩容时，要先知道 100G 那块在磁盘的什么位置

任务 B：给机器做体检（6 条命令）

```
cat /etc/os-release
nproc
free -h
df -h /
ip -c addr show
docker ps
```

预期看到什么：

命令	看懂哪个数字	健康的样子
cat /etc/os-release	VERSION=	26.04
nproc	一个整数	≥ 4（虚拟机建议给 4~6 核）
free -h	Mem: 行的 available 列	≥ 12G（24G 里分给 Ubuntu 的部分）
df -h /	Avail 列	≥ 60G
ip -c addr show	非 lo 网卡的 inet 那行	形如 192.168.x.x 或 10.x.x.x
docker ps	报什么错	现在报 command not found 是 正常的 （Docker 还没装，第 2 课才装）；这条是第 2 课验收项 3、4 的伏笔

我的实测输出（贴这里）

（待你贴回）

我的解读

（待你贴输出后填写）

卡点

（待记录）

本课必背清单（学完必须带走的东西）

复习只看这一节，不用重读整课。三档要求：★脱手 = 闭卷能敲；可查 = 知道有它、用时查参数；认识 = 看到能说出它干什么。

★ 必须脱手（闭卷能敲出骨架，7 条）

命令	一句话：它是干什么的	自测
cat /etc/os-release	看发行版和版本号	☐
uname -r	看内核版本	☐
lsblk -f	看磁盘 / 分区 / 文件系统 / 挂载点	☐
df -h	看每个分区还剩多少空间	☐
free -h	看内存， 重点看 available 列	☐
systemd-detect-virt	判断物理机 / 虚拟机 / 容器	☐
ip -c a	看网卡和 IP	☐

可以查（知道存在，用时查参数）

- df -i —— 看 inode（文件数量）有没有用完：**空间没满却写不进文件**就是它
- lsuf +L1 —— 找已删除但被进程占着句柄的文件（磁盘满排障的关键一跳）
- nproc —— CPU 核数；lsblk -d -o NAME,ROTA,SIZE —— ROTA=0 固态、1 机械
- dmidecode -s system-product-name —— 判断是不是云主机（需 root）

认识即可（看到能说出它干什么）

- /proc/cpuinfo（逐核明细）、/proc/1/cgroup（容器线索）
- buff/cache 的含义：空闲内存被内核拿去做磁盘缓存

必须能说出口的 3 句话

- 接手一台机器先**取证**，不改任何配置（os-release → uname -r → nproc/free/df → detect-virt → ip → systemctl --failed）。
- free -h 要看 **available**，不是 free —— 空闲内存被内核拿去做磁盘缓存了，free 小是正常的。
- df -h 看**空间**，df -i 看 **inode**，**两个都会满**，而且现象完全不同。

本课要记进笔记的三件事

- systemd-detect-virt 一条命令就能判断虚拟化平台，比翻设置快

- free -h 要看的是 available 而不是 free（为什么，下节课讲）
- df -h 看空间，df -i 看 inode —— 两个都会满，这是后面排障的伏笔

第 2 课 • 装工具 + Docker

对应计划：W1 D1 (1.1 工具清单) + W1 D2 (1.2 Docker / 镜像加速 / 日志轮转) **节奏建议：**分两个晚上。**今晚只做任务 A；**任务 B~E 明晚做。**目标：**工具装齐、Docker 能用，并亲手踩一次**"装完 Docker 还要重新登录"**的坑。

这一课在解决什么

排障靠的是工具，不是记忆。所以 1.1 的清单要一次装齐 —— 第 3 周排 CPU/内存/磁盘故障用 `htop/iotop/sysstat`，第 2 月抓包用 `tcpdump`，第 3 月写脚本要用 `shellcheck` 检查质量。

Docker 是第 4 月整套 K8s 实验的地基。这一课的重点不是"装上"，而是中间那个著名坑：**加进 docker 组之后，不重新登录，docker ps 照样报权限错误**。我们故意踩一次再修好 —— 自己踩过的坑才记得住。

任务 A：装工具（对应 1.1）

```
sudo apt update && sudo apt install -y \
  curl wget git vim tmux tree jq unzip shellcheck \
  net-tools dnsutils iputils-ping traceroute mtr-tiny \
  tcpdump telnet netcat-openbsd \
  htop iotop sysstat lsof strace \
  build-essential python3 python3-pip python3-venv \
  ca-certificates gnupg lsb-release
```

装完必须补两步（不做第一步，sysstat 等于白装）：

```
# ① sysstat 默认不采数据；不打开，以后 sar 查历史永远是空的，你会以为它坏了
sudo sed -i 's/^ENABLED="false"/ENABLED="true"/' /etc/default/sysstat
sudo systemctl enable --now sysstat

# ② 验证三件事
systemctl is-active sysstat      # 期望 active
sar -u 1 3                      # 期望能打出 CPU 使用率
which shellcheck tmux jq        # 期望三条路径都打印出来
```

`iotop` 必须 `sudo iotop` 才能看到全部进程的 IO —— 普通用户只能看到自己的。
`nmap` 不在必装清单：它是扫描工具，**只能扫自己的资产**；扫别人网段在国内属违法行为。
如果 `apt update` 慢到影响心情，可以换国内源（第 2 月有专门的换源课）。注意 Ubuntu 24.04 起默认源文件是 `/etc/apt/sources.list.d/ubuntu.sources`（deb822 格式），**旧教程里的 `/etc/apt/sources.list` 可能已经不存在**，别照着旧教程改。

任务 B：装 Docker（官方 apt 源）

```
# ① 加官方 GPG 公钥（apt 靠它验证“这个包真的来自 Docker”）
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# ② 加官方 apt 源（$(...) 自动取你的系统代号，不用手写）
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# ③ 安装
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# ④ 验证“服务本身是好的”
sudo systemctl is-active docker
sudo docker run --rm hello-world
```

为什么不用 `curl -fsSL https://get.docker.com | sudo sh`？那等于把网上一个随时会变的脚本直接喂给 root 执行。能用、快，但不可审计、不可复现。**面试问“你怎么装 Docker”，答官方 apt 源是加分项。**

任务 C（本课重点）：docker 组权限坑，故意踩一次

第 1 步：先确认“服务是好的，是我没权限”

```
sudo docker ps      # 能输出空表 → daemon 正常
docker ps           # 报 permission denied while trying to connect to the Docker daemon socket
```

两条命令的反差告诉你：**报错说的是“权限”，不是“服务没起来”**。排障第一步永远是分清这两类。

第 2 步：把当前用户加进 docker 组

```
sudo usermod -aG docker $USER
```

第 3 步：不重新登录，立刻试 ← 坑在这里

```
docker ps          # 期望：还是 permission denied（别慌，这是对的）
id -nG             # 期望：里面没有 docker
getent group docker # 期望：这一行末尾有你的用户名
```

为什么：usermod 改的是 /etc/group 这份"名单"；而一个进程属于哪些组，是**登录那一刻**就写进进程凭证的。你现在的 shell 是登录时创建的，名单改完它也不知道。

第 4 步：修好（二选一）

```
newgrp docker      # 临时：另开一个带 docker 组的 shell，退出即失效
# 或：完全退出登录（图形界面注销 / ssh 断开重连）—— 推荐，最干净

docker ps          # 期望：正常输出空表，不再报错
id -nG             # 期望：这次里面出现 docker
```

安全认知（面试会问）：docker 组等价于 root。能起特权容器的人，把宿主机 / 挂进容器就拿到了 root shell。所以生产上"给某人加 docker 组"是发一张 root 通行证，不是"方便一下"。

任务 D：镜像加速 + 日志轮转（/etc/docker/daemon.json）

```
sudo mkdir -p /etc/docker
sudo tee /etc/docker/daemon.json > /dev/null <<'EOF'
{
  "registry-mirrors": [
    "https://<你的专属ID>.mirror.aliyuncs.com"
  ],
  "log-driver": "json-file",
  "log-opts": { "max-size": "50m", "max-file": "3" }
}
EOF
sudo systemctl daemon-reload && sudo systemctl restart docker
docker info | grep -A3 "Registry Mirrors"    # 验证加速器生效
```

专属加速地址怎么拿（免费）：阿里云控制台 → 容器镜像服务 ACR → 创建**个人实例** → 左侧「镜像加速器」→ 复制你**账号专属的** https://xxxx.mirror.aliyuncs.com。这串 ID 绑定你的账号，**不能抄别人的**，也别用公共地址（dockerproxy.com 已失效；mirror.ccs.tencentyun.com **只在腾讯云内网稳定**）。备选：docker.m.daocloud.io、docker.mirrors.ustc.edu.cn。

三条规则：

1. **只配 1 主 1~2 备**。Docker 按顺序试，源不通要等超时才切下一个，堆多了只会更慢。
2. **registry-mirrors 只代理 pull，不代理 push。**
3. **log-opts 只对新建容器生效**，已存在的容器不会变 —— 以后查"磁盘被日志吃满"要记得这条。

故意搞坏一次（安全的小演练）：把 daemon.json 里随便删掉一个逗号或引号，然后

```
sudo systemctl restart docker      # 会失败
systemctl status docker            # 看到 failed
sudo journalctl -xeu docker | tail -20 # 期望能看到 JSON 解析相关的错误原文
# 改回去，再 restart，恢复
```

这就是"看报错原文 → 定位 → 验证"的最小闭环，第 1 月的排障反射从这一课开始练。

任务 E：本课验收（对应 1.7 的第 3、4、5 条）

```
docker ps          # 免 sudo 不报错 → 验收 3 通过
docker run --rm hello-world      # 打印 Hello from Docker! → 验收 4 通过
docker run -d -p 8080:80 nginx    # 起一个 nginx
curl -I localhost:8080 | head -1  # 期望 HTTP/1.1 200 OK → 验收 5 通过
docker rm -f $(docker ps -q --filter ancestor=nginx) # 收尾：删掉它
```

双系统用户注意：**验证服务永远在 Ubuntu 里用 curl，不要在 Windows 里开浏览器。**两个系统不能同时开；而且服务器上本来就没有浏览器，curl 才是生产里的方式。

我的实测输出（贴这里）

（待你贴回：每个任务的输出整段贴，报错更有价值）

我的解读

（待你贴输出后填写）

卡点

（待记录）

本课必背清单（学完必须带走的东西）

复习只看这一节。三档要求：★脱手 = 闭卷能敲；可查 = 知道有它、用时查参数；认识 = 看到能说出它干什么。

★ 必须脱手（闭卷能敲出骨架，8 条）

命令	一句话：它是干什么的	自测
sudo apt update / sudo apt install -y <包>	刷新索引 / 装包	☐
systemctl enable --now <服务>	立刻启动 + 开机自启（只 start 不 enable，重启就没了）	☐
sudo usermod -aG docker \$USER	把当前用户加进 docker 组	☐
id -nG	看当前进程真正生效的组有哪些	☐
docker ps	看正在运行的容器	☐
docker run -d -p 8080:80 nginx	后台起容器，宿主 8080 → 容器 80	☐
docker logs <容器> / docker rm -f <容器>	看日志 / 强制删除	☐
curl -I localhost:8080	只看响应头验证服务（HEAD 请求）	☐

可以查（知道存在，用时查参数）

- dpkg -S \$(which nginx) —— 命令属于哪个包；dpkg -L <包> —— 包装了哪些文件
- journalctl -xeu docker —— 看服务失败的真正原因（status 只说病了，它才说哪儿病了）
- sudo ss -lntp —— 看端口被谁占、监听在哪个地址
- docker system df / docker builder prune —— 看 / 清 Docker 磁盘占用
- docker info | grep -A3 "Registry Mirrors" —— 验证镜像加速是否生效
- apt-mark hold <包> —— 锁住版本不让他升级

认识即可（看到能说出它干什么）

- daemon.json 的三个关键项：registry-mirrors、log-opts、live-restore
- overlay2（联合文件系统）、containerd / runc（Docker 架构链路）
- newgrp docker —— 临时换组的办法

必须能说出口的 4 句话

1. 安装 ≠ 生效：程序装上 ≠ 服务在跑 ≠ 数据在采（sysstat 不 enable 就没有历史数据）。
2. 组关系在登录那一刻写进进程凭证：usermod -aG 只改 /etc/group 名单 → 必须重新登录；所以 getent group docker 里有你、id -nG 里没有。
3. systemctl status 只说病了，journalctl -xeu 才说哪儿病了。
4. docker 组 ≈ root：能起特权容器、能挂宿主 /，所以加 docker 组是发一张 root 通行证。

本课要记进笔记的三件事

- 组权限的生效时机：usermod -aG 只改名单（/etc/group），已登录的 shell 不会立刻拿到新组 —— 要重新登录或 newgrp
- 分清两类报错：permission denied（权限问题）和 service not running（服务问题），排查方向完全不同
- daemon.json 是纯 JSON，不能写注释；改完必须 daemon-reload + restart，且 log-opts 只对新建容器生效

第 3 课 • 让服务能被连上（SSH + 固定 IP）

对应计划：W1 D2（1.2④ SSH + 固定 IP）+ 1.7 验收第 7 条 节奏建议：分两个晚上。今晚任务 A~C（把服务装起来、能连上、免密），明晚任务 D~F（原理 + 加固 + 排障演练）。目标：本机能连自己、外部能连你、密钥登录可用，并且知道改 sshd 的安全顺序（顺序错了就是把自己锁在门外）。

这一课在解决什么

前面两课是"认清机器"和"把工具装好"，这一课开始让机器能被人从外面操作。

为什么非要它不可：

- 第 3 月写批量巡检脚本，要 ssh 到多台机器取数据；
- 第 5 月从云服务器验证你本地服务，要靠 SSH 反向隧道；
- 第 6 月CI/CD 自动部署，本质就是"脚本替你 ssh 上去执行"。

没有 SSH，这三件事全部断掉。而且 ssh 是运维每天敲得最多的命令，也是最容易一不小心把自己关在门外的命令 —— 所以这一课的真正目标是：既能连上，又只有你能连上。

任务 A：装好 SSH 服务，并让它开机自启

```
sudo apt install -y openssh-server
sudo systemctl enable --now ssh

systemctl status ssh --no-pager | head -5      # 期望 active (running)
sudo ss -lntp | grep ':22'                     # 期望看到 0.0.0.0:22 或 :::22 在 LISTEN
ssh localhost                                  # 会让你输密码，登进去算通（1.7 验收第 7 条）
```

预期与分支（这一段是本课排障主线的种子）：

现象	属于哪一类问题	往下查什么
ss 里看不到 22 端口	服务没起来	journalctl -u ssh -n 30
ssh localhost 报 Connection refused	端口/监听问题	服务状态 + 监听地址
ssh localhost 报 Permission denied	认证问题	密码/密钥、/var/log/auth.log

注意 enable --now 是两条动作合一：enable = 开机自启，--now = 立刻启动。只 start 不 enable，重启一次服务就没了——这正是第 2 课"安装 ≠ 生效"的延续。

任务 B：记下 IP，并搞清楚"到底谁能连进来"

```
ip -c a                # 记下非 lo 网卡上的 inet
hostname -I            # 省事的做法（只看地址）
systemd-detect-virt    # 回顾第 1 课：我是虚拟机还是物理机
```

这一步才是本课最容易卡住的地方，因为"连不上"往往不是 SSH 的问题，而是网络形态的问题：

你的形态	谁能连到你的 22 端口	要做什么
虚拟机 + NAT（默认）	只有宿主机（还得配端口转发）	在虚拟化软件里把宿主 2222 转发到虚机 22，然后 ssh -p 2222 user@127.0.0.1
虚拟机 + 桥接	同一局域网的所有设备	记下 IP；在虚拟化软件/路由器里把它固定住
双系统（物理机）	局域网设备	进路由器做 DHCP 保留（推荐）或在系统里配静态 IP

为什么必须固定 IP（这条比你想象的贵）：

IP 一变，你写好的巡检脚本、~/.ssh/config、Ansible 清单、监控配置、防火墙放行规则全部失效——而且是那种"昨天还好好的，今天全报错"的失效，最难查。第 3 月你会真的被它咬一次。

双系统用户：优先用路由器的 DHCP 保留（在路由器里绑定 MAC → IP），比在 Ubuntu 里写静态 IP 简单，也不会因为写错配置而断网。

```
# 想看看本机网络怎么配的（Ubuntu 22.04+/24.04+ 用 netplan）
ls /etc/netplan/
```

⚠ 不要在虚拟机里随手抄网上的 netplan 配置：改错的后果是直接断网，远程改的话当场失联。虚拟机用户动手前先打快照。

任务 C：密钥登录（免密）

```
ssh-keygen -t ed25519 -C "w1-ubuntu-lab"  # 学习机可以一路回车（不设 passphrase）
ls -l ~/.ssh/                             # 看生成了什么
cat ~/.ssh/id_ed25519.pub                 # 这是公钥，可以公开
ssh-copy-id localhost                    # 把公钥送到服务器（也可 user@<IP>）
ssh localhost                             # 这次应该不再问密码
```

为什么用 ed25519 而不是 RSA：

	ed25519	RSA
密钥长度	256 位就够安全	至少 3072 位
速度/体积	快、签名短	慢、体积大
现状	现代默认选择	只在老设备兼容时才用

生产上私钥必须设 passphrase，再用 ssh-agent 免去重复输入：eval \$(ssh-agent) && ssh-add。
id_ed25519（私钥）永远不外传，id_ed25519.pub（公钥）才是给服务器的。

免密登录的原理（本课最关键的 5 步，面试主线）

用门禁卡打比方，但记住每一步在防谁：

- 1. 客户端说："我是 user，我想用这把公钥（指纹 xxx）登录"
- 2. 服务器去 ~/.ssh/authorized_keys 里找这把公钥——没有就拒绝
- 3. 找到后，服务器发一个随机挑战串给客户端
- 4. 客户端用私钥对这个挑战签名，把签名发回去
- 5. 服务器用公钥验证签名——验过才放行

关键点：私钥全程不离开你的机器，网络上跑的是签名。所以它比密码安全——密码要么在网络上传输，要么在服务器上以哈希形式存放；而私钥根本不出门，服务器也不需要"知道"任何能冒充你的东西。

权限为什么必须是 700 / 600（真实高频坑）

```
ls -ld ~/.ssh
ls -l ~/.ssh/authorized_keys
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys  # 不对就修
```

对象	权限	为什么是它
~/.ssh	700	别人能读你的目录，就能替换里面的 authorized_keys
~/.ssh/authorized_keys	600	别人能写它，就能塞进自己的公钥——等于直接拿到你的账号
~/.ssh/id_ed25519	600	私钥被读到 = 身份被冒用
家目录	不能 group/other 可写	家目录可写 = 能把整个 .ssh 换掉

sshd 的 StrictModes yes（默认）就是干这件事的：**权限不对 → 直接拒绝密钥登录**，客户端只看到一句 Permission denied (publickey)，不告诉你为什么 —— 所以这个坑很难自己猜出来。

任务 D：把 SSH 连接拆成四段（面试主线）

这一段是本课的核心知识。任何 SSH 报错，先归到下面四段之一，再往下查：

阶段	发生什么	挂了报什么
① TCP 连接	连到对端 22 端口	Connection refused（端口没监听/被拒） / timed out（网络或防火墙丢包）
② 版本协商 + 密钥交换	协商算法，Diffie-Hellman 交换出 会话密钥	算法不匹配（老客户端 ↔ 新服务端）
③ 主机验证	客户端比对 known_hosts 里的服务器公钥	Host key verification failed（换了机器/重装/中间人）
④ 用户认证	密码，或公钥签名挑战	Permission denied (publickey/password)

known_hosts 的坑：重装系统 / 换了 IP 归属的机器之后
ssh-keygen -R localhost # 删掉旧指纹（先确认是“真的换了机器”，不是有人中间人）

为什么不能养成“直接删 known_hosts 了事”的习惯：那等于把**防中间人**这道门自己拆了。正确做法是先核对服务器指纹（ssh-keyscan 或从控制台看）。

任务 E：加固（本课最容易把自己锁在外面的一步）

先看现状，再动手：

```
sudo sshd -T | grep -E 'permitrootlogin|passwordauthentication|pubkeyauthentication|port'
systemctl cat ssh | grep -iE 'KillMode|ExecReload'        # 看看这个发行版的 sshd 是怎么被 systemd 管的
```

改配置的安全顺序（血泪顺序，顺序错了要进机房插显示器）：

- 1. 留后路：另开一个终端并 sudo -s 保持登录，不要关它
- 2. 备份：sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
- 3. 改完先语法检查：sudo sshd -t（必须无输出才算过）
- 4. 用 reload 而不是 restart：sudo systemctl reload ssh
- 5. 新开一个窗口验证密钥能登录，确认成功之后才关掉旧窗口

```
sudo sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sudo sed -i 's/^#\?PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sudo sshd -t                                # 无输出 = 语法通过
sudo systemctl reload ssh
```

⚠ 在关掉密码登录之前，必须先确认密钥登录真的能用（任务 C 验证过）。顺序反了 = 把自己锁在门外。这不是吓唬人，是生产上真实发生过事故，严重的要去机房插显示器。

顺序口诀：留后路 → 语法检查 → reload → 新窗口验证 → 才关旧窗口。

先搞清 reload 和 restart 的区别，别背结论：reload 是让主进程重读配置、不断开已有连接；restart 是停掉再起。有些发行版的 sshd 单元设了 KillMode=process，所以 restart 也不会掐断你现有的会话 —— 但这是**发行版实现细节**，不能假设所有机器都这样。上面那条 systemctl cat 就是让你在**自己机器上看清真相**（这就是第 1 课的母题 M3：让机器自己说话）。

对外暴露面的控制：

- 云服务器：安全组**不要对 0.0.0.0/0** 长期开放 22，只放行你的固定出口 IP；
- 本机：sudo ufw status，开了 ufw 记得放行 22；
- 防暴力破解（了解即可，第 5 月有云机器再练）：fail2ban，登录失败 N 次自动封 IP。

面试加分点：改 SSH 端口只能**减少扫描噪音**，不是**安全措施**（扫描器照样扫全端口）。真正的安全来自密钥认证 + 最小暴露面 + 封禁策略。

任务 F：排障演练（本课重头戏，动手做）

从外到内五层，每层一条命令：

```
# ① 网络可达吗
ping -c 2 <IP>
# ② 端口通吗（别只看 ssh 的报错）
nc -zv <IP> 22                                # 或 telnet <IP> 22
# ③ 服务在跑吗、监听在哪
systemctl status ssh --no-pager | head -3
sudo ss -lntp | grep ':22'
# ④ 认证为什么失败（服务端的真相）
sudo tail -30 /var/log/auth.log
# ⑤ 客户端看到什么细节
ssh -vvv user@<IP> 2>&1 | tail -40
```

两个必背的排障动作：

- **服务端**：临时把 LogLevel 调到 DEBUG3 再 reload，然后 sudo tail -f /var/log/auth.log —— 服务端日志才会说出真正的原因；

• **客户端**: `ssh -vvv` —— 能看出卡在哪一段 (TCP 没通? 密钥没加载? 公钥被拒?)。

三个"故意搞坏"的小练习 (安全, 因为本机密码登录还在) :

1. `chmod 644 ~/.ssh/authorized_keys` → 密钥登录失败 → 在 `auth.log` 里找提示 → 改回 600
2. 在 `authorized_keys` 里故意改掉一个字符 → 失败 → 改回
3. 在 `sshd_config` 里写一行错的 → 被 `sshd -t` 拦下 —— **亲眼看到"语法检查"救了你一次**

可选进阶: 端口转发 (第 5 月的伏笔)

```
# 本地转发 -L: 把远程的 3306 映射到本机 13306 (安全访问内网数据库的真实用法)
ssh -N -L 13306:127.0.0.1:3306 user@<跳板或数据库主机>

# 反向隧道 -R: 把本机 80 推到公网服务器的 10080
ssh -N -R 10080:localhost:80 root@<公网IP>
```

第 5 月一定会用到 `-R`: 因为云服务器在公网, **永远无法主动访问你 NAT 后面的私网 IP**; 反向隧道是零成本、零第三方的解法。

我的实测输出 (贴这里)

(待你贴回: 任务 A~F 每步的输出, 报错照原样贴)

我的解读

(待你贴输出后填写)

卡点

(待记录)

本课必背清单 (学完必须带走的东西)

复习只看这一节。三档要求: ★脱手 = 闭卷能敲; 可查 = 知道有它、用时查参数; 认识 = 看到能说出它干什么。

★ **必须脱手 (闭卷能敲出骨架, 7 条)**

命令	一句话: 它是干什么的	自测
<code>sudo systemctl enable --now ssh</code>	启动 SSH 服务 + 开机自启	☐
<code>systemctl status ssh</code>	看服务状态	☐
<code>sudo ss -lntp</code>	看 22 端口在不在监听、监听在哪个地址	☐
<code>ssh localhost</code>	最小连通性验证用例	☐
<code>ssh-keygen -t ed25519</code>	生成密钥对	☐
<code>ssh-copy-id <user@host></code>	把公钥送到服务器	☐
<code>sudo sshd -t</code>	改完 sshd 配置先做语法检查 (无输出 = 通过)	☐

可以查 (知道存在, 用时查参数)

- `ssh -vvv user@host` —— 客户端最详细的排障输出 (看卡在哪一段)
- `sudo tail -30 /var/log/auth.log` —— 服务端看到的认证真相
- `nc -zv <IP> 22` —— 只测端口通不通 (**别只看 ssh 的报错**)
- `ssh-keygen -R <host>` —— 删掉 `known_hosts` 里的旧指纹
- `sudo sshd -T` —— 打印当前**实际生效**的 `sshd` 配置
- `ssh -N -R 10080:localhost:80 root@<公网IP>` —— 反向隧道 (第 5 月必用)

认识即可 (看到能说出它干什么)

- `netplan (/etc/netplan/)`、`hostname -I`、DHCP 保留、`fail2ban`
- `sshd_config` 里的 `StrictModes`、`GatewayPorts`

必须能说出口的 4 句话

1. **SSH 连接四段**: ① TCP 连 22 → ② 版本协商 + 密钥交换 → ③ 主机验证 (`known_hosts`) → ④ 用户认证。任何 **SSH 报错**先归到哪一段, 再往下查。
2. **免密登录的本质**: 服务器发随机挑战 → 客户端用**私钥签名** → 服务器用**公钥验签**; **私钥永远不出本机**。
3. **改 sshd 的安全顺序**: 留后路 (保持一个 `root shell`) → `sshd -t` 语法检查 → `reload` → **新窗口验证** → 才关旧窗口。
4. 权限必须 `~/.ssh = 700`、`authorized_keys = 600` —— `StrictModes` 会让权限不对的密钥**静默失败** (只报 `Permission denied (publickey)`)。

本课要记进笔记的三件事

- **SSH 连接四段**: TCP → 密钥交换 → 主机验证 → 用户认证。看到报错先归到哪一段, 再往下查
- **免密登录的本质**: 服务器发挑战 → 客户端用**私钥签名** → 服务器用**公钥验签**; **私钥永远不上网**
- **改 sshd 的安全顺序**: 留后路 → `sshd -t` 语法检查 → `reload` → 新窗口验证 → 才关旧窗口

面试题库: [第 3 课 · SSH + 固定 IP](#) · [面试题库](#)

待开始

第 4 课 • 建立"搞坏了能回来"的底气 (备份与快照)

预告： `/etc` 配置备份 + 快照 / Timeshift。这一课必须在任何"故意搞坏"的练习之前上完，否则第 2 周改坏 `sudoers` 就是真事故。